

Les données



Les variables

- Une variable est déclarée dès qu'elle est utilisée
- Tous les noms de variables sont précédés d'un \$, suivi par une lettre ou un _ et puis des lettres, des chiffres ou des _
(ex : \$var = 5 ;)
- Le type de la variable se détermine en fonction de son contenu et peut évoluer au fur et à mesure de son utilisation
- Les identificateurs sont sensibles à la casse
- Des alias de variables (références) peuvent être définis en préfixant le nom de la variable de &
(ex. \$aliasNom=&\$nom)



Les variables prédéfinies



- PHP fournit des variables prédéfinies qui peuvent être utilisées dans les scripts. Cela concerne aussi bien les variables qui seront passées par un formulaire en ligne que des variables de session, de cookies HTTP... ou des variables de serveur et d'exécution.
- La grande majorité de ces variables correspond à des tableaux contenant les informations souhaitées.

```
<?php
echo $_SERVER['REMOTE_ADDR']."<BR>";
echo $_SERVER['SERVER_NAME']."<BR>";
echo $_SERVER['HTTP_USER_AGENT']."<BR>";
?>
```

80.14.202.31

www.stri.ups-tlse.fr

Mozilla/5.0 (Windows; U; Windows NT 5.1; rv:1.7.3) Gecko/20040913 Firefox/0.10.1



Les constantes



- Les constantes sont définies avec la fonction **define**.
(ex. **define** ('AUTEUR', 'toto'))
- Les noms des constantes ne commencent pas par \$
- Par convention, les identificateurs de constantes sont en majuscules
- Seuls les types de données scalaires peuvent être placés dans une constante (bool, entier, double et string)
- Si une constante est utilisée avant qu'elle ne soit définie, elle prend comme valeur la chaîne de caractères correspondant à son nom.
- Utilisez la fonction **defined()** pour savoir si une constante existe ou pas.

```
<?php
define("CONSTANTE", "Bonjour le monde.");
echo CONSTANTE; // affiche "Bonjour le monde."
echo Constante; // affiche "Constante" et une note.
?>
```



Les constantes prédéfinies



`__FILE__` nom du fichier en cours d'exécution
`__LINE__` ligne du fichier en cours d'exécution
`E_ERROR` message de l'erreur survenue
`E_WARNING` message d'avertissement
`E_NOTICE` note d'interprétation du fichier
`E_ALL` tous les messages d'erreur réunis
`TRUE` vrai
`FALSE` faux
`NULL` valeur nulle
`PHP_VERSION` version de PHP
`PHP_OS` nom de l'OS utilisé



Type boolean



```
<?php
    $foo = True; // assigne la valeur TRUE à $foo
    if ($action == "show_version") {
        echo "La version est la 1.23";
    }

    // Ceci n'est pas nécessaire
    if ($show_separators == TRUE) {
        echo "<hr>\n";
    }

    // car vous pouvez simplement utiliser
    if ($show_separators) {
        echo "<hr>\n";
    }
?>
```

les valeurs 0 ou 0.0 , la valeur false ,
la chaîne vide, la chaîne "0" , un
tableau vide, la valeur NULL sont
assimilées à *faux*

Les constantes TRUE et FALSE
sont insensibles à la casse

-1 est considéré comme vrai



Types integer et float



```
<?php
$a = 1234; # nombre décimal
$a = -123; # nombre négatif
$a = 0123; # nombre octal (équivalent à 83 décimal)
$a = 0x1A; # nombre hexadécimal (équivalent à 26 décimal)
```

?>

```
<?php
$a = 1.234;
$b = 1.2e3;
$c = 7E-10;
```

?>



Type string (1/6)



➤ Les chaînes de caractères peuvent être définies de 3 façons:

- délimitation avec des '
- délimitation avec des "
- utilisation de la syntaxe HereDoc

■ Définition avec '

- peut contenir des " et des retours à la ligne
- caractères spéciaux supportés : \' (') et \\ (\)

■ Définition avec "

- peut contenir des ' et des retours à la ligne
- les variables sont *expansées*, i.e remplacées par leur valeur
- caractères spéciaux: \n (retour à la ligne) \r (retour chariot)
\t (Tabulation) \\ (caractère \)
\\$ (caractère \$) \" (caractère ")



Type string (2/6)



■ Définition avec HereDoc

- permet d'affecter un contenu tenant sur plusieurs lignes
- début de la définition : <<< identificateur
- fin de la définition

- * identificateur (le même que celui utilisé au début) au **tout début** d'une ligne
- * ne placer aucun caractère ensuite sur la même ligne, à part un ; éventuellement
- * les variables sont expansées

```
echo <<< EOT
Mon nom est $nom
Mon prénom est $prenom
EOT;
```

```
<?php
$boisson = 'vin';
echo "Du $boisson, du pain et du fromage!";
// OK, car "," n'est pas autorisé dans les noms de variables
echo "Il a goûté plusieurs $boissons";
// Pas OK, car 's' peut faire partie d'un nom de variable, et PHP recherchera alors $boissons
echo "Il a goûté plusieurs ${boisson}s"; // Remarquez les accolades
// OK
?>
```



Type string (3/6)



➤ Il existe de nombreuses fonctions de traitement de chaînes:

- **strlen(\$s)** : longueur de \$s
- **ltrim(\$s)**, **trim(\$s)**, **rtrim(\$s)** : suppression des espaces
- **strtoupper(\$s)**, **strtolower(\$s)** : modification de la casse
- **strcmp(\$s1, \$s2)** : comparaison de chaînes
- **strpos(\$s1, \$s2)** : Trouve la première occurrence d'un caractère dans une chaîne (équivalent à **strstr()**)



Type string (4/6)



- **strpos (\$s1, \$s2) :** Trouve la position d'un caractère dans une chaîne

Cette fonction peut retourner **FALSE**, mais elle peut aussi retourner une valeur équivalent à **FALSE**, utilisable dans une condition if simple.
Utilisez l'opérateur **===** pour tester la valeur exacte de cette fonction.

```
<?php
$mystring = 'abc';
$findme = 'a';
$pos = strpos($mystring, $findme);
// Notez l'utilisation de ===. Un simple == ne donnerait pas le résultat escompté;
// car la lettre 'a' est à la position 0 (la première).
if ($pos === false) {
    echo "La chaîne '$findme' n'a pas été trouvée dans la chaîne '$mystring'";
} else {
    echo "La chaîne '$findme' a été trouvée dans la chaîne '$mystring'";
    echo " et à la position $pos";
}
?>
```



Type string (5/6)



- **Accès et modification des caractères d'une chaîne**

Les caractères d'une chaîne sont accessibles et modifiables en spécifiant leur offset (le premier caractère est d'offset 0) entre accolades, après le nom de la variable

```
<?php
// Lit le premier caractère de la chaîne
$str = 'Ceci est un test.';
$first = $str{0};
// Lit le troisième caractère de la chaîne
$third = $str{2};
// Lit le dernier caractère de la chaîne
$str = 'Ceci est un test.';
$last = $str{strlen($str)-1};
// Modifie le dernier caractère de la chaîne
$str = 'Ceci est un test.';
$str{strlen($str)-1} = '!';
?>
```

➤ Conversion de chaînes de caractères

Lorsqu'une chaîne de caractère est évaluée comme une valeur numérique, le résultat et le type de la variable sont déterminés comme suit :

- La chaîne de caractères est de type "double" si elle contient un des caractères '.', 'e' ou 'E'. Sinon, elle est de type entier ("integer").

- La valeur est définie par la première partie de la chaîne. Si la chaîne de caractères débute par une valeur numérique cette valeur sera celle utilisée. Sinon, la valeur sera égale à 0 (zéro).

```
<?php
$foo = 1 + "10.5";           // $foo est du type double (11.5)
$foo = 1 + "-1.3e3";         // $foo est du type double (-1299)
$foo = 1 + "bob-1.3e3";      // $foo est du type integer (1)
$foo = 1 + "bob3";           // $foo est du type integer (1)
$foo = 1 + "10 petits cochons"; // $foo est du type integer (11)
$foo = "10.0 cochons " + 1;   // $foo est du type double (11)
$foo = "10.0 cochons " + 1.0; // $foo est du type double (11)
?>
```

```
<?php
$bool = TRUE; // un booléen
$str = "foo"; // une chaîne de caractères
$int = 12; // un entier
echo gettype($bool); // affiche "boolean"
echo gettype($str); // affiche "string"
// Ceci est un entier, on l'incrmente de 4
if (is_int($int)) {
    $int += 4;
}
}
?>
```

Conversions de types (1/2)

- Lorsque dans une expression deux variables sont de types différents, une conversion implicite de l'une des variables est effectuée selon des règles de conversions.
- Un exemple de convertisseur automatique de type est l'opérateur '+'. Si un des opérandes est de type double, alors tous les opérandes sont évalués comme des variables de type double et le résultat est de type double. Sinon, tous les opérandes sont évalués comme des variables de type entier et le résultat sera du type entier. Il est à noter que cela NE CHANGE PAS le type des opérandes. Le seul changement est la manière dont les opérandes sont évaluées.

```
<?php
$foo = "0"; // $foo est une chaîne de caractères (ASCII 48)
$foo += 2; // $foo est maintenant du type entier (2)
$foo = $foo + 1.3; // $foo est maintenant du type double (3.3)
$foo = 5 + "10 Petits cochons"; // $foo est du type entier (15)
$foo = 5 + "10 cochonnets"; // $foo est du type entier (15)
?>
```

Conversions de types (2/2)

- On peut forcer le type d'une variable en utilisant:
bool `settype` (mixed var, string type);
/* mixed est utilisé dans le cours pour indiquer qu'il peut y avoir différent type */

```
<?php
$foo = "5bar"; // string
$bar = true; // boolean
settype($foo, "integer"); // $foo is now 5 (integer)
settype($bar, "string"); // $bar is now "1" (string)
?>
```

- On peut forcer le type d'une variable en faisant un cast (comme en C):

```
<?php
$foo = 10; // $foo is an integer
$bar = (boolean) $foo; // $bar is a boolean
?>
```




Type array (1/6)



- En PHP, un tableau est une liste de valeurs accessibles par une clé ou indice. On a alors une association clé/valeur.
Les tableaux en PHP sont donc des tableaux associatifs. Dans le cas où la clé est numérique, on peut parler de tableaux numériques.
- Dans un même tableau, les valeurs peuvent être de types différents. De même, on peut avoir des clés de types différents.
- La taille d'un tableau peut évoluer durant l'exécution du script
==> **Tableau Dynamique**
- Un tableau array peut être créé avec la fonction array().



Type array (2/6)



- La fonction array peut prendre en argument des structures key => value, séparées par des virgules.

```
<?php
$arr = array("foo" => "bar", 12 => true);
echo $arr["foo"]; // bar
echo $arr[12];    // 1
?>
```

- La fonction list() permet d'affecter les valeurs contenues dans un tableau à une liste de variables (ex. list(\$var1, \$var2, ...) = \$tab;)



Type array (3/6)



- On peut aussi affecter directement les valeurs au tableau.

```
$tab[0] = 1 ;  
$tab[1] = "toto" ; # on peut mélanger les contenus  
$tab["toto"] = "titi" ; # on peut mélanger les clés  
$tab["toto"][8] ; # tableau à deux dimensions  
$tab["toto"][8]["encore"][2] ; # quatre dimensions  
$suite = array(1, 2, 3, 4) ;  
$personne = array("type" => "M.", "nom" => "Smith") ;  
list($num1, $num2) = $suite ; # $num1 vaut 1 et $num2 vaut 2
```

- On peut connaître le nombre d'éléments d'un tableau grâce aux fonctions :
 - `sizeof()` : retourne le nombre d'éléments d'un tableau, ou
 - `count()` : retourne le nombre d'éléments d'un tableau s'il existe, 1 si la variable n'est pas un tableau et 0 si la variable n'existe pas.



Type array (4/6)



- Si vous omettez une clé lors de la spécification d'un tableau, l'indice maximum + 1 sera utilisé comme clé par défaut. Si aucun indice numérique n'a été généré, ce sera 0. Si vous en spécifiez une qui a déjà été assignée, la nouvelle valeur écrasera la précédente.

```
<?php  
// Ce tableau est identique à  
array(5 => 43, 32, 56, "b" => 12);  
// Celui ci  
array(5 => 43, 6 => 32, 7 => 56, "b" => 12);  
?>
```

- On peut avoir un tableau associatif de tableaux indicés

```
$caddy = array(  
    "fruits" => array("pomme", "banane", "raisin"),  
    "legumes" => array("salade", "poivron"),  
    "divers" => array("chocolat", "beurre")  
);  
$article = $caddy["fruits"][1]; // "banane"
```



Type array (5/6)



- Chaque tableau entretient un pointeur courant qui sert à naviguer en son sein grâce aux fonctions :

- reset() : place le pointeur interne sur le **premier élément** et retourne sa valeur,
- current() : retourne la valeur de l' **élément courant** ,
- next() : place le pointeur interne sur l' **élément suivant** et retourne sa valeur,
- prev() : place le pointeur interne sur l' **élément précédent** et retourne sa valeur,
- each() : retourne la **paire clé/valeur courante** du tableau et avance le pointeur sur l'élément suivant (c'est la seule fonction qui ne retourne pas faux si l'élément vaut 0 ou ""). Si aucun élément n'est accessible, each(...) renvoie false

```
<?php
$transport = array('pied', 'velo', 'voiture', 'avion');
$mode = current($transport); // $mode = 'pied';
$mode = next($transport);    // $mode = 'velo';
$mode = next($transport);    // $mode = 'voiture';
$mode = prev($transport);    // $mode = 'velo';
$mode = end($transport);     // $mode = 'avion';
?>
```



Type array (6/6)



- Un tableau peut être trié en utilisant les fonctions suivantes :

- asort()/arsort() : trie le tableau en ordre croissant/décroissant de **valeurs** ,
- ksort()/ksort() : trie le tableau en ordre croissant/décroissant de **clés** ,
- sort()/rsort() : trie le tableau en ordre croissant/décroissant **clés et valeurs** (on perd la correspondance clé/valeur),
- uasort()/uksort()/usort() : trie le tableau de la même façon que leurs quasi-homonymes (u pour *user*) mais avec une fonction de comparaison fournie par l'utilisateur.

```
<?php
$fruits = array( "d"=>"papaye",
                "a"=>"orange",
                "b"=>"banane",
                "c"=>"ananas");

asort($fruits);
for(reset($fruits); $key = key($fruits); next($fruits)) {
    echo "fruits[$key] = ".$fruits[$key]."\n";
}
?>
```